Taller de Programación

Información General para la Entrega

- **Modalidad:** El presente entregable debe ser resuelto de forma estrictamente individual. Cualquier indicio de colaboración, intercambio de código o similitud excesiva entre entregas será motivo de anulación inmediata y reporte a las autoridades correspondientes.
- **Fecha de entrega:** 10 de abril del 2026 a las 11:50am.
- **Uso de Inteligencia Artificial:** Está terminantemente prohibido el uso de herramientas de generación de código (como ChatGPT, Claude, GitHub Copilot, entre otras). El objetivo fundamental de esta evaluación es que el estudiante desarrolle su propia lógica de programación y capacidad de resolución de problemas.
- **Verificación de Autoría:** El docente se reserva el derecho de realizar entrevistas técnicas aleatorias. Si se detecta que el estudiante es incapaz de explicar la lógica de su código o si se utiliza material avanzado no cubierto en el programa oficial (como funciones lambda, comprensión de listas, decoradores o librerías externas no solicitadas), la calificación se verá afectada significativamente, pudiendo ser de 0.
- **Formato:** Entrega de un único archivo fuente de Python siguiendo el formato de nombre `NombreApellidoLab#.py`. El código debe estar debidamente comentado, explicando de manera sucinta la lógica detrás de cada bloque o estructura de control implementada.

Laboratorio Evaluativo: Sistema de Monitoreo de Presión Industrial

Bienvenidos, ingenieros. Hoy se les ha asignado la responsabilidad de supervisar el sistema de calderas de nuestra planta de energía. La gerencia ha detectado que los registros de presión se están perdiendo porque los programas actuales no guardan la información de forma permanente. Su misión es construir un sistema robusto que lea los datos de los sensores, los procese siguiendo estrictas normas de seguridad y genere un informe de auditoría.

Para este proyecto, utilizaremos un archivo de entrada llamado `registros.txt`, el cual contiene el nombre de cada caldera en una línea y su nivel de presión actual en la línea inmediatamente inferior.

```
Caldera-Norte
120
Caldera-Sur
250
Reactor-Principal
80
Turbina-Alfa
300
```

Para organizar este sistema, deben diseñar una estructura que represente a cada sensor. Es vital que esta estructura proteja la integridad de los datos, asegurando que el valor de la presión no pueda ser manipulado accidentalmente por errores en otras partes del código o con valores físicamente imposibles, como presiones negativas.

El sistema debe contar con mecanismos internos que validen cualquier intento de cambio en el valor de la presión antes de que este se guarde en la memoria. Para el diseño, utilicen el nombre de clase `SensorPresion` y asegúrense de que el atributo que guarda el valor numérico sea totalmente **privado**, utilizando la variable `presion`.

Además de la información individual de cada caldera, la planta necesita llevar un control estadístico global. Deben implementar una forma de contabilizar cuántas lecturas se han realizado en total a través de la variable `total_lecturas`, y definir un umbral de seguridad general para toda la planta llamado `LIMITE_PELIGRO`, el cual debe estar fijado en **200 PSI**. Este límite no debe pertenecer a una sola caldera, sino ser una regla compartida por todos los sensores de la red.

El flujo del programa debe iniciar abriendo el archivo de registros y procesando los datos por parejas:

- primero el nombre y
- luego el valor numérico

Asegurándose de limpiar cualquier residuo de texto invisible o saltos de línea al leer. Una vez que todos los objetos estén creados y almacenados en una lista, el sistema debe recorrerlos para verificar cuáles han superado el límite de peligro. Al finalizar, el programa no solo debe mostrar un resumen en pantalla, sino que debe generar un nuevo archivo físico llamado `alertas.txt` donde se documenten únicamente los nombres de las calderas que se encuentran en estado crítico, garantizando que esta información no se pierda al apagar la computadora.

Especificaciones Técnicas y Funcionalidades

Para que el motor de este sistema funcione correctamente, la estructura de su diseño debe incluir funciones específicas encargadas de tareas únicas.

- Primero, la clase debe contar con una función especial encargada de recibir los datos externos (nombre y presión) al momento de nacer cada objeto. Esta función debe ser capaz de inicializar los atributos y, al mismo tiempo, disparar el mecanismo de validación que mencionamos anteriormente para asegurar que la presión inicial sea correcta. No olviden que dentro de este mismo bloque es donde debe incrementarse el contador global de la planta cada vez que una nueva caldera entra al sistema.
- Además, es fundamental implementar dos funciones gemelas que actúen como la única puerta de acceso al valor de la presión. La primera función debe tener como única misión devolver el valor para que el programa principal pueda consultarlo sin modificarlo. La segunda función debe actuar como un filtro de seguridad; debe recibir un nuevo valor, verificar si es válido (que no sea negativo) y solo entonces actualizar la memoria interna del objeto.
- Fuera de la clase, el programa debe estar orquestado por una serie de pasos lógicos. Deben desarrollar una sección del código encargada exclusivamente de la lectura del archivo de texto. Esta parte debe ser capaz de abrir la "puerta" del archivo de forma segura y utilizar un ciclo para extraer la información en bloques. Como los datos vienen escalonados (una línea para el nombre y otra para el número), su lógica debe estar preparada para leer dos veces dentro de cada vuelta del ciclo, asegurando que el valor numérico sea transformado de texto a un número entero antes de crear el objeto.
- Finalmente, deben crear una lógica de salida que procese la lista de objetos recolectados. Esta sección debe tener la capacidad de evaluar cada caldera individualmente y decidir si su nombre debe ser enviado al archivo de auditoría. Para esto, deben abrir un archivo de salida en modo de escritura y redactar un encabezado profesional seguido de los nombres de las calderas en peligro. El éxito del laboratorio dependerá de que el programa sea capaz de cerrar todos los archivos correctamente y mostrar un mensaje final que confirme cuántas calderas fueron procesadas exitosamente.

Guía de Interacción y Simulación del Sistema

La interacción con el programa será automatizada: una vez que el código se ejecute, este buscará el archivo `registros.txt` en la misma carpeta donde se encuentra su script de Python. Es responsabilidad de ustedes asegurarse de que el archivo exista y contenga datos válidos antes de iniciar la simulación.

Ejemplo de Flujo de Ejecución (Simulación)

Imaginen que su archivo `registros.txt` contiene los datos de la **Caldera-Sur** con una presión de **250**. Al correr el programa, sucederá lo siguiente:

1. **Apertura Silenciosa:** El programa abre el archivo y lee el nombre "Caldera-Sur" y el número "250".
2. **Nacimiento del Objeto:** Se crea una instancia de `SensorPresion`. En ese microsegundo, el contador global `total_lecturas` sube de 0 a 1.
3. **Filtro de Seguridad:** El valor "250" intenta entrar al atributo privado. El mecanismo de validación lo deja pasar porque es un número positivo.
4. **Evaluación de Riesgo:** El programa compara los **250 PSI** de la caldera contra el `LIMITE_PELIGRO` de **200 PSI**.
5. **Registro de Alerta:** Al detectar que supera el límite, el programa prepara el nombre "Caldera-Sur" para escribirlo en el archivo final.

Resultados Esperados en Consola

Si su lógica es correcta, al finalizar la ejecución deberían ver un resumen profesional en la terminal similar a este:

```
--- SISTEMA DE MONITOREO INDUSTRIAL ---
Leyendo registros de presión...
Analizando: Caldera-Norte | Estado: Seguro
Analizando: Caldera-Sur | Estado: ¡PELIGRO!
Analizando: Reactor-Principal | Estado: Seguro
Analizando: Turbina-Alfa | Estado: ¡PELIGRO!

[AUDITORÍA] Se han generado alertas en el archivo 'alertas.txt
[RESUMEN] Total de sensores procesados: 4.
----------------------------------------
```

Verificación de Persistencia

La prueba final de que su sistema funciona no está en la pantalla, sino en su carpeta de trabajo. Tras cerrar el programa, debe aparecer el archivo `alertas.txt`. Al abrirlo, deberían encontrar una lista limpia y organizada:

```
REPORTE DE INCIDENCIAS - CALDERAS CRÍTICAS
------------------------------------------
1. Caldera-Sur
2. Turbina-Alfa
```

Hoja de Evaluación: Sistema de Monitoreo de Presión Industrial

Categoría	Criterio de Evaluación	Puntaje Máximo	Logrado (S/N)	Puntos Obtenidos
Arquitectura POO	Define la clase `SensorPresion` con el atributo privado `__presion`.	15 pts		
	Implementa `@property` para la lectura y `@presion.setter` para la validación (bloqueo de negativos).	15 pts		
	Utiliza Atributos de Clase para `LIMITE_PELIGRO` (200) y `total_lecturas`.	10 pts		
Manejo de Archivos	Abre y lee `registros.txt` línea por línea (sin usar split o métodos no vistos).	15 pts		
	Genera el archivo `alertas.txt` con el formato y contenido solicitado.	10 pts		
	Implementa `with open` para garantizar el cierre automático de ambos archivos.	10 pts		
Lógica y Control	Convierte correctamente el texto a entero (`int`) y limpia espacios (`strip`).	5 pts		
	Almacena los objetos en una lista y los recorre para filtrar las alertas.	10 pts		
Calidad de Código	El código compila/ejecuta sin	10 pts		

	errores y los nombres son significativos (snake_case).			
TOTAL		**100 pts**	**NOTA:**	